

Evaluating Expressions – Step by Step

Example: `10 - 2 * (1 + 3) + "x" + 7 / 2`

1 Resolve explicit parentheses

Innermost `( )` first – use steps 2–3 inside it.

`(1 + 3) = 4 → 10 - 2 * 4 + "x" + 7 / 2`

2 Add the invisible parentheses

Higher in the pyramid → binds first.

Same level → follow the associativity.

`((10 - (2 * 4)) + "x") + (7 / 2)`

3 Evaluate one operator at a time

- `int / int → int`, cut off: `7 / 2 = 3`
- `% → remainder`: `8 % 3 = 2`
- `int with double → double`
- a `String` operand → `+` concatenates

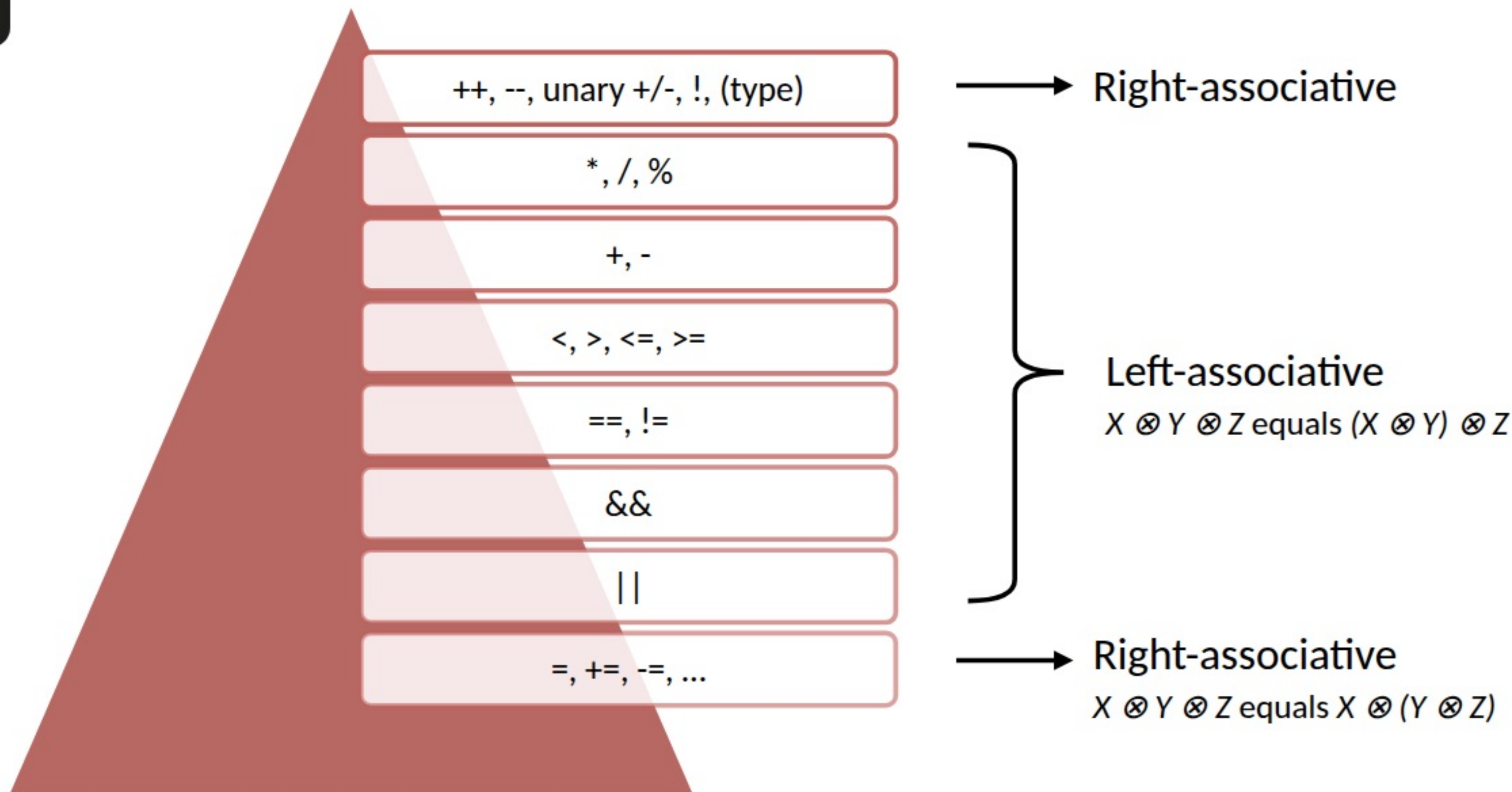
`2*4=8 → 10-8=2 → 2+"x"="2x"`
`→ 7/2=3 → "2x"+3="2x3"`

4 Write down the result

A `double` is always printed with a decimal point: `4.0`

Output: `2x3`

Note: This recipe works for expressions without side effects. With `++`, `--` or `=` inside an expression, remember that Java always evaluates operands **left to right**.



Operators that share the same precedence level always have the same associativity.

Precedence & associativity set the parentheses, **types** decide what each operator does.
When in doubt: **use parentheses!**